development_history

MIDAS GUI — Development History

A commit-by-commit record of what changed and when, so you can (a) find **when** a particular behaviour/feature entered the code and (b) know **what to roll back** to undo a specific effect.

> Keep this file updated when you add commits: append a new entry at the bottom of the chronological log and add a row to the quick-reference index. Regenerate the PDF the same way as the other docs (pandoc + headless Chrome) if you keep one.

How to use this document

- **Find when a change landed:** search the change → commit index below, or `git log --oneline -- <path>` for a specific file, or `git log -S"<code string>"` to find the commit that introduced/removed a string.
- **See a commit's exact diff:** `git show <hash>` (whole commit) or `git show <hash> -- <path>` (one file).
- **Undo one commit cleanly (keeps later history):** `git revert <hash>`. For a range: `git revert <oldest>^..<newest>`.
- **Undo just one file back to a commit:** `git checkout <hash> -- <path>` then commit. To see the file *as of* a commit without changing anything: `git show <hash>:<path>`.
- **Hard reset to a point (destructive — local only, never on pushed history):** `git reset --hard <hash>`. Prefer `git revert` on a shared branch.
- **Dependencies matter.** Many later commits build on earlier ones (e.g. the unified data-loader, the config overlay). The “Roll back” note on each entry flags when reverting will disturb dependents.

Branch: `main`. All commits are authored by Dishant Beniwal (AI pair-authored). Dates are commit dates (YYYY-MM-DD).

Change → commit quick-reference index

Packaging / build / launch

Change	Commit
Initial v1.0.0 release (9-tab app, all backends)	`f19ee8c`
setuptools.build_meta backend (broader pip compat)	`3cd1f36`
`launch.py` standalone launcher	`2e9f419`
Startup crash diagnostics + robust launcher (Windows)	`916ece2`
MIDAS-style packaging (LICENSE, pyproject, release.sh, smoke tests)	`e862135`
Ship test_data sample data in the repo (fresh clone works out of the box)	`7e157e0`
Drop midas_suite from env (unblock conda env create; backends via -e .)	`72a1770`
Pin environment.yml to verified NumPy-1 set + README recreate steps	`68313f8`

Data Viewer (Tab 0)

Change	Commit
Radial-integration plot, beam-centre ring picking, zoom persistence	`acf0866`
Zoom/pan preserved across frames (autoRange off)	`d9e527e`
Intensity-range mask, calibration-file loading, click-to-draw ring	`d50b588`
Compact 3-per-row lattice + cubic quick-set; mask default max(99.99pct,1e5)	`41f28f5`
Projection Skip-frames field; compact geometry/intensity fields; Lsd separators	`108ea7d`
Intensity-statistics panel + histogram (bottom of loader)	`2b7a695`
Tilt/distortion-aware radial integration when a calibration is loaded	`df544d2`

Change	Commit
Data Viewer Top-N brightest pixels + mask-aware stats + full-geometry receive	`bc86d74`
Calibrate per-coefficient distortion, frame averaging, seed feedback	`b0a5cc3`
Batch read-only “Calibration values” card	`b6f1681`
Pump Probe publication plotting, delay colour-bar, default detector mask	`49623ae`
Gitignore local context, internal docs, large sample sets	`b251ca8`
Live Data card: in-tab EPICS PVA stream via pvapy, reload button	`1769f69`
Live viewer: manual colormap/level changes now survive new frames	`234ded9`
Wider image/radial-plot splitter handle	`69f470c`
Radial profile: Y-min defaults to 0.9x data min, manual Y-range sticks	`96f8add`
Simulate rings is now a live toggle (auto-recomputes on param change)	`ecf6d01`
Live PV field becomes a device dropdown (Preferences ▸ Devices)	`aa82cb6`
Fix image-view autorange drift; bound pan/zoom to image	`c156c62`
Radial profile plot: bound pan/zoom to the data extent	`cde4bb2`
Ring simulation ty/tz tilt fields; configurable spin-box step sizes	`5b8e3b3`
Tilt-aware profile w/o calibration file; ring 2θ-cutoff/thickness/live-button fixes; save-calibration buttons; radial-plot X-floor + zoom persistence	`e14c1ea`
Native-menu-backed A/M manual axis limits (radial plot + histogram); Top-N “I >” intensity floor	`3c15ae7`
Multi-material ring simulation (per-row checkbox/color/name, Material dialog)	`0e9ed21`

Change	Commit
Load-calibration card moved below Simulate button; loads now sync ty/tz too; Intensity-range title bar is the on/off checkbox	`05ce224`
Live Data “Use Buffer” last-N-frames ring buffer (Projection/stack analysis on live data)	`911f8ac`
Fix: lock live ring buffer against GUI/worker-thread race	`a88ba1f`
Fix: refresh-timer starvation during fast live streaming	`2cfd9cd`
Fix: cap live buffer to 100 frames (memory bound)	`839770d`
Fix: “All frames” stats computed off the GUI thread	`08392c6`
Fix: debounce intensity-mask pixel≤/pixel> spinbox edits	`57ced2f`
Fix: warn when composite mask silently drops a shape-mismatched source	`81b8ea8`
Sim Detector: hardware-free fake PVA stream in the Live Data dropdown	`3d96cb1`
Image viewer: persist manual color-scale window (incl. histogram zoom) across live frames	`3b12fbf`
Fix: “Use Buffer” state now carries into Start instead of resetting	`f3a1b91`
Unrestrict λ/Lsd/pixel-size ranges; tighten decimals; Rings/Labels/thickness on one row	`2525277`
“?” help button explaining radial-integration (R-bin) calculation	`b7a1518`

Mask Builder (Tab 1)

Change	Commit
Unbounded σ fields; independent spatial/temporal auto-mask; Viewer↔Calibrate geometry hand-off	`10df828`

Change	Commit
Temporal-constancy accepts a 3-D HDF5 (time,y,x) stack	`2385f75`

Calibrate (Tab 2) / Refinement (Tab 3)

Change	Commit
Abort buttons; dark/bright/background field correction; auto-load on browse	`7d010d7`
Calibration input accepts paramstest/poni/json	`41f28f5`
Load-calibration-file (geometry + λ) helpers	`b381d8d`
Multi-column paramstest results readout + Send-to-Data-Viewer button	`455ca4e`
Calibrate Results all-text (drop distortion table; named distortion, bigger font)	`3a6ecb9`
Fix calibrate() initial_BC_y TypeError (kwargs filter) + pin scikit-image	`b1642fa`
Pick-Ring fit overlay recolored blue (was same amber as simulated rings)	`3eb4a44`
“Corrected” rings drawn synchronously from fitted ty/tz (drop background worker)	`5b8e3b3`

Batch Integrate (Tab 4)

Change	Commit
Live folder MONITOR + per-file legend	`1149afc`
Clear results + inline per-file labels & plot controls	`524f5f1`
Stacked profiles: publication themes, point+line, symbol size, x-units, grid	`d135c80`
Waterfall R/2θ/Q x-axis selector	`2385f75`

PDF Analysis (Tab 6)

Change	Commit
Polyatomic midas_pdf reduction pipeline + vendored midas_pdf + calibration loading	`b381d8d`
Defaults to real I(Q) test data	`1149afc`

Pump Probe (TR-XRD) tab

Change	Commit
New tab: TRR pooling + MIDAS-engine integration + ΔI(q,delay) + 4 pyqtgraph views + publication-quality plots + default MIDAS calibration	`590b410`

Cross-cutting UI / infrastructure

Change	Commit
Linux visible files/folders style fix	`fc754d0`
Pixel-weighted azimuthal mean; readable QMenu; no-scroll combos	`41f28f5`
Unified Data-Loader panel + draggable 3-panel splitter (tabs 0/2/3/4)	`aa9395e`
Clickable λ label with K-edge foil menu	`b628446`
Clickable px label with detector pixel-size menu	`3248638`
Per-user JSON configuration system + Preferences dialog	`d30be2c`
Modular tab visibility (show/hide optional tabs)	`590b410`
Multi-profile config (Preferences ▸ Profile row)	`5b8e3b3`
File ▸ Save/Load GUI State (all 10 tabs, with mask/calibration sidecars)	`5b8e3b3`
User-adjustable interface scaling (HiDPI / 4K, QT_SCALE_FACTOR)	`d5fafd8`
Default optional tabs reduced (Corrections/PDF/Texture/Export hidden)	`7e157e0`

Change	Commit
Lsd displayed in mm (calculations & calibration files stay in μm)	`c4e1c12`
Fix “Populating font family aliases” warning (real fixed-width fonts)	`d6df1f8`
Fix fresh-env launch crash — colormap resolution without matplotlib	`6332b3d`
Fix native Bus-error crash on Run Calibration — cap BLAS/OMP thread pools	`0b4326d`
Preferences ▸ Devices tab; Live PV field becomes a device dropdown	`aa82cb6`
Widen non-data-derived numeric field caps (tilts to ±180°, iteration/geometry/hyperparameter fields to effectively unbounded) across Data Viewer/Calibrate/Corrections/Mask/Refine/Batch	`53f8f19`

Stability / performance / consistency (review-driven, phases 1–3)

Change	Commit
Clean worker shutdown, drop QThread.terminate(), guard stdout, offload projection	`399f3d7`
Waterfall $O(N^2) \rightarrow O(N)$, frame-slider debounce, radial-grid + stats caching	`4462ee1`
Dedupe distortion map + paramstest writer, align calibrant lattices, texture colormap	`8846619`

Documentation

Change	Commit
README not-on-PyPI clone/conda instructions	`cd05ced`
gui_documentation.md added	`2e5f7c9`
gui_documentation.pdf added	`75c135c`
README/environment install via `pip install midas_suite`	`aceb40f`

Change	Commit
implementation_details.md/.pdf added	`524f5f1`
config_gui.md + config.example.json added	`d30be2c`
development_history.md/.pdf added	`6d6f3fb`

Chronological commit log (oldest → newest)

`f19ee8c` — Initial release: midas_gui v1.0.0 (2026-06-30)

Effect: First version. Nine-tab PyQt5 GUI over `midas_calibrate_v2` / `midas_integrate_v2`: mask building, auto-calibration, refinement, batch integration + waterfall, corrections, PDF, texture, export. Entry points `midas-gui` / `python -m midas_gui`. **Files:** entire `midas_gui/` package + `pyproject.toml`, `README`, `environment.yml`. **Roll back:** this is the root — everything depends on it; don't revert.

`3cd1f36` — setuptools.build_meta backend (2026-06-30)

Effect: Build backend switched for setuptools≥61 compatibility. **Files:** `pyproject.toml`. **Roll back:** `git revert 3cd1f36` (only affects builds).

`2e9f419` — add launch.py standalone launcher (2026-06-30)

Effect: `python /path/to/launch.py` runs the app from anywhere. **Files:** `launch.py`. **Roll back:** safe to revert; later hardened by `916ece2`.

`fc754d0` — Linux style fix for invisible files/folders (2026-06-30)

Effect: Stylesheet fix so file/folder text is visible on Linux. **Files:** `midas_gui/style.py`. **Roll back:** `git revert fc754d0` (cosmetic).

`acf0866` — Data Viewer: radial plot, ring picking, zoom persistence (2026-06-30)

Effect: Added the radial-integration plot, beam-centre ring picking, and zoom-persistence across a stack in the Data Viewer; widened left panels. **Files:** `tab_view.py` (+1-line width bumps in other tabs). **Roll back:** revert to remove the Data Viewer radial plot/ring picking.

`d9e527e` — preserve zoom/pan across frames (2026-06-30)

Effect: Disabled pyqtgraph autoRange on redisplay so zoom/pan survive frame changes. **Files:** `widgets.py`. **Roll back:** revert if you *want* auto-refit per frame.

`d50b588` — Data Viewer: intensity mask, calib loading, click-to-draw ring (2026-07-01)

Effect: 99.9-pct intensity-range mask default; load calibration (json/poni/ paramstest); click-to-draw ring; test-data defaults point at local `test_data/`. **Files:** `tab_view.py`, `widgets.py`, `helpers.py`, `constants.py`, `tab_mask.py`. **Roll back:** revert to drop these Data Viewer features (note `helpers.py` geometry parsing added here is reused later — see `b381d8d`).

`916ece2` — startup crash diagnostics + robust launcher (2026-07-01)

Effect: Logs uncaught exceptions/native faults to `~/midas_gui_error.log`, prevents PyQt slot-exception aborts, isolates failing tabs, hardens the launcher. **Files:** `app.py`, `launch.py`. **Roll back:** revert only if you want raw exceptions to propagate; not recommended.

`7d010d7` — Abort buttons + dark/bright/background + auto-load (2026-07-01)

Effect: Abort on calibrate/batch; dark/bright/background field correction (file/folder/HDF5, index-range averaging, divide/subtract); browse auto-loads, “Load” buttons removed; compact FieldSelector. **Files:** `tab_batch/calibrate/corrections/mask/pdf/refine/texture/view.py`, `widgets.py`, `workers.py`, `helpers.py`. **Roll back:** wide-reaching; reverting removes field correction + auto-load across tabs. The Abort behaviour here was later re-worked by `399f3d7` (no `terminate()`).

`e862135` — MIDAS-style packaging (2026-07-01)

Effect: BSD-3 LICENSE, MIDAS-convention `pyproject.toml`, `release.sh` + `RELEASING.md`, headless `tests/` smoke suite. **Files:** `LICENSE`, `README.md`, `RELEASING.md`, `pyproject.toml`, `release.sh`, `tests/`. **Roll back:** revert affects packaging/tests only.

`41f28f5` — compact lattice, pixel-weighted azimuthal mean, readable menus (2026-07-06)

Effect: Data Viewer 3-per-row lattice + cubic quick-set; intensity-mask default max(99.99pct,1e5); **pixel-weighted azimuthal mean** (selectable vs legacy η-bin mean) fixing off-detector-BC distortion; batch calibration accepts paramstest/poni/ json; readable right-click menus; no-scroll combos. **Files:** `helpers.py`, `workers.py`, `tab_view/batch/calibrate/pdf/refine/texture.py`, `widgets.py`, `style.py`. **Roll back:** to undo the azimuthal-averaging change specifically, prefer editing the `weighted` default rather than reverting the whole commit (it bundles UI changes).

`cd05ced` — docs: not-on-PyPI install instructions (2026-07-06)

Effect: README clone+conda+run instructions; relative env self-install. **Files:** `README.md`, `environment.yml`. **Roll back:** docs only.

`2e5f7c9` — docs: add gui_documentation.md (2026-07-06)

Effect: First committed user guide. **Files:** `documentation/gui_documentation.md`. **Roll back:** docs only.

`75c135c` — docs: add gui_documentation.pdf (2026-07-06)

Effect: PDF build of the guide. **Files:** `documentation/gui_documentation.pdf`. **Roll back:** docs only.

`aceb40f` — docs: install via `pip install midas_suite` (2026-07-06)

Effect: Corrected backend-install instructions. **Files:** `README.md`, `environment.yml`. **Roll back:** docs only.

`aa9395e` — Unified Data-Loader panel + 3-panel splitter (tabs 0/2/3/4) (2026-07-07)

Effect: Shared `DataLoaderPanel` (Data/Dark/Bright/Background/Mask, per-mode frame controls) + `MaskSelector`; corrections applied on tabs 0/2/3/4; each of those tabs restructured into a draggable `[Data Loader | Parameters | Display]` splitter. **Files:** `widgets.py` (+442), `tab_view/calibrate/batch/refine.py` (large rewrites), `app.py`, `helpers.py`, `workers.py`, docs. **Roll back: high impact** — this is the base for the Batch/Calibrate/Refine and later Pump Probe layouts. Reverting breaks the 3-panel tabs and the Pump Probe tab's left panel. Avoid a blanket revert; fix forward instead.

`b381d8d` — Tab 6 polyatomic midas_pdf pipeline + calibration loading (2026-07-08)

Effect: PDF tab rewritten to the total-scattering pipeline; **vendored `midas_pdf`** under `midas_gui/_vendor/` + `pdf_backend.py`; `helpers` `geometry_fields_from_file` / `result_ns_from_geometry_file` (paramstest/json/poni) used by Calibrate and PDF "Load calibration file…". **Files:** `_vendor/midas_pdf/**` (new, large), `pdf_backend.py`, `tab_pdf.py`, `workers.py`, `helpers.py`, `tab_calibrate.py`, `constants.py`, `pyproject.toml`, docs. **Roll back:** reverting removes the PDF pipeline **and** the shared geometry-file loaders that later tabs (incl. Pump Probe via `spec_from_geometry_file`) rely on — revert with care.

`1149afc` — Batch live MONITOR + legend; PDF real-data default (2026-07-08)

Effect: MONITOR button (stream mode) watches a folder and integrates new frames via `FolderMonitorWorker`, reusing a cached detector map (factored `build_integration_context()` + `write_profile()`, shared with `BatchWorker`); per-file legend; PDF tab defaults to real I(Q). **Files:** `tab_batch.py`, `workers.py`, `widgets.py`, `constants.py`, `tab_pdf.py`, docs. **Roll back:** to remove

MONITOR only, revert this commit — but `build_integration_context()` introduced here is **reused by the Pump Probe worker** (`590b410`), so a full revert would break Pump Probe integration.

`524f5f1` — Batch Clear results + inline labels; implementation-details doc (2026-07-09)

Effect: "Clear results" button; inline per-file curve labels + line-width/font toolbar on stacked profiles; new `implementation_details.md/.pdf`. **Files:** `tab_batch.py`, `widgets.py`, `documentation/implementation_details.*`. **Roll back:** safe, localized to Batch + the new doc.

`10df828` — Mask unbounded σ + split auto-mask; Viewer↔Calibrate geometry (2026-07-09)

Effect: Removed σ upper limits; split statistical auto-mask into independent Spatial-outlier / Temporal-constancy checkboxes; geometry hand-off `DataViewer.pushGeometry` / `Calibrate.pullGeometry` → `apply_geometry`. **Files:** `tab_mask.py`, `tab_view.py`, `tab_calibrate.py`, `app.py`, `workers.py`, docs. **Roll back:** revert to restore bounded σ / combined auto-mask / no geometry hand-off.

`d135c80` — Batch stacked-profiles: themes, point+line, x-units, grid (2026-07-09)

Effect: `StackedProfileViewer` publication/dark themes, point+line, symbol-size, R/2θ/Q x-unit selector, grid toggle. (This viewer's styling is the template the Pump Probe plots later mirror.) **Files:** `widgets.py` (+228), `tab_batch.py`, docs. **Roll back:** localized to the Batch stacked-profile viewer.

`2385f75` — Batch waterfall R/2θ/Q axis; Mask 3-D HDF5 stack (2026-07-10)

Effect: Waterfall x-unit selector via a converting `AxisItem` + `_convert_radial()`; Mask temporal-constancy reads a single 3-D (time,y,x) HDF5. **Files:** `widgets.py`, `tab_batch.py`, `tab_mask.py`, `workers.py`, docs. **Roll back:** `_convert_radial` / `_UnitAxis` introduced here are reused by the Pump Probe heatmap — don't fully revert if Pump Probe is present.

`b628446` — clickable λ label with K-edge foil menu (2026-07-10)

Effect: Compact clickable "λ" label → K-edge foils menu (sets λ from edge energy). `constants.K_EDGE_FOILS`, `HC_KEV_A`, `helpers.make_kedge_label`. **Files:** `constants.py`, `helpers.py`, `tab_view/calibrate/pdf.py`, docs. **Roll back:** localized; `make_pixel_label` (next) reuses the factored helper.

`108ea7d` — Data Viewer projection Skip-frames + compact fields (2026-07-10)

Effect: Projection "Skip frames" field; narrower Axis/Skip/pixel fields; Lsd thousands-separators. **Files:** `tab_view.py`, docs. **Roll back:** localized.

`3248638` — clickable px label with detector pixel-size menu (2026-07-10)

Effect: Clickable “px” label → GE/Varex/Pilatus/Eiger sizes; `constants.PIXEL_PRESETS`; factored `helpers._clickable_menu_label`. **Files:** `constants.py`, `helpers.py`, `tab_view.py`, `tab_calibrate.py`, docs. **Roll back:** localized.

`2b7a695` — Data Viewer intensity-statistics panel + histogram (2026-07-10)

Effect: `IntensityStatsPanel` (histogram + p70/p90/p99/p99.9/p99.99 with pixel counts) pinned to the loader bottom (stack mode); Current/All/projected scope. **Files:** `widgets.py`, `tab_view.py`, docs. **Roll back:** localized.

`d30be2c` — per-user configuration system + Preferences dialog (2026-07-12)

Effect: Per-user JSON config overlays shipped defaults at import; `settings.py`; `prefs_dialog.py` (Geometry/Paths/Materials/Calibrants/Menus/Algorithms); Settings menu; `DEFAULT_KERNEL/PIPELINE/OUTPUT_FORMAT/ERROR_MODEL/COLORMAP`; tab combos honor configured defaults. New `config_gui.md`, `config.example.json`, `tests/test_config.py`. **Files:** `settings.py` (new), `prefs_dialog.py` (new), `constants.py` (+165), `app.py`, `tab_batch.py`, `tab_calibrate.py`, `widgets.py`, docs, tests. **Roll back: high impact** — the config overlay + `DEFAULT_*` and the Preferences dialog are extended by the modular-tabs commit (`590b410`). Reverting this would also break `ui.visible_tabs` and the Tabs preferences section.

`399f3d7` — Stability: clean worker shutdown, no terminate(), stdout guard (2026-07-12)

Effect: `closeEvent` stops all tab QThreads; removed every `QThread.terminate()` (cooperative interrupt + orphan instead); `CalibrationWorker` restores stdout only if still its own; Data Viewer projection moved to `ProjectionWorker` (off GUI thread). **Files:** `app.py`, `tab_batch.py`, `tab_calibrate.py`, `tab_view.py`, `workers.py`. **Roll back:** reverting reintroduces the exit hard-crash risk and UI-freeze on projection — not recommended. Phase 1 of the 3-part review.

`4462ee1` — Performance: waterfall O(N), debounce, caching (2026-07-12)

Effect: Waterfall pre-allocated buffer + 100 ms coalesced redraw ($O(N)$ not $O(N^2)$); 60 ms frame-slider debounce; cached radial bin-index grid; skip all-frame stats on frame-only changes. **Files:** `tab_view.py`, `widgets.py`. **Roll back:** reverting slows large scans / scrubbing; behaviour otherwise same. Phase 2 of the review.

`8846619` — Consistency: dedupe maps/writer, align lattices (2026-07-12)

Effect: `helpers._PARAMSTEST_DISTORTION` derived from `constants._V2_TO_V1` (removed duplicated `_V2V1` dicts); shared `helpers.write_standalone_paramstest` used by Calibrate + Export; LaB6/Si `_LATT/_LC` aligned to `MATERIALS`; texture colormap honors `DEFAULT_COLORMAP`. **Files:** `constants.py`, `helpers.py`, `tab_calibrate.py`, `tab_export.py`, `tab_texture.py`. **Roll back:** reverting reintroduces duplicated code paths; low functional risk. Phase 3 of the review.

`590b410` — Modular tabs + Pump Probe (TR-XRD) tab + plot quality (2026-07-12)

Effect: (1) **Modular tab visibility** — Data Viewer/Mask/Calibrate/Batch always shown, the rest toggled in Settings ▸ Preferences ▸ Tabs (`ui.visible_tabs`, applied live); `app.apply_tab_visibility()`. (2) **New Pump Probe tab** (`tab_pumpprobe.py`) — TRR filename pooling, MIDAS-engine integration via new `PumpProbeWorker` (reuses `build_integration_context` / `integrate_frame`), ΔI(q,delay), three-panel layout, four pyqtgraph views; ships a converted MIDAS `paramstest` as the default calibration. (3) **Plot quality** — white publication theme, black axes/labels/titles, readable legends, ΔI colorbar, aligned reference panel, shared draw-mode + line/point/font-size toolbar, bounded pan/zoom. **Files:** `tab_pumpprobe.py` (new), `workers.py` (+`PumpProbeWorker`), `app.py`, `constants.py`, `prefs_dialog.py`, `tests/test_smoke.py`, docs (`gui_documentation`, `config_gui`, `config.example.json`). **Roll back:** to remove **only Pump Probe**, delete `tab_pumpprobe.py` and its wiring in `app.py`/`workers.py`; to remove **only modular tabs**, revert the `app.apply_tab_visibility` / `prefs_dialog` Tabs section and `constants` tab lists. A full `git revert 590b410` removes both features together. Depends on `aa9395e` (DataLoaderPanel), `1149afc` (build_integration_context), `2385f75` (`_convert_radial`/`_UnitAxis`), `d30be2c` (config/prefs).

`6d6f3fb` — docs: add development_history.md/.pdf (2026-07-13)

Effect: Added this per-commit change log + change→commit index + rollback guide. **Files:** `documentation/development_history.md`, `documentation/development_history.pdf`. **Roll back:** docs only.

`d5fafd8` — UI scaling: user-adjustable whole-interface zoom (HiDPI / 4K) (2026-07-13)

Effect: Whole-application scale (layout + fonts) for HiDPI / 4K monitors. `constants.DEFAULT_UI_SCALE` (config `ui.ui_scale`); `app.main()` applies it via `QT_SCALE_FACTOR` **before** the QApplication is created (so fixed widths, splitters, pyqtgraph and stylesheet px all scale uniformly) + `AA_UseHighDpiPixmaps`. Manual control in **Preferences ▸ Display** (spin + 100/125/150/200 % presets) and **Settings ▸ Interface scaling…**; both persist `ui.ui_scale` and offer an immediate self-restart (`MainWindow.restart_app` via `QProcess`) since the factor is read only at startup. **Files:** `constants.py`, `app.py`, `prefs_dialog.py`, docs (`gui_documentation`, `config_gui`, `config.example.json`). **Roll back:** `git revert d5fafd8`. Self-contained (depends only on the config system `d30be2c`); reverting removes the scale control and the app renders at 1.0×. To keep the feature but disable it, set `ui.ui_scale` to 1.0.

`c4e1c12` — Display Lsd in mm (calculations & files stay in µm) (2026-07-13)

Effect: The sample-to-detector distance is entered/shown in **mm** in the Data Viewer, the Calibrate seed, and Preferences ▸ Geometry; conversion to/from µm happens only at the display boundary (`DataViewerTab._lsd_um()`, ×1000 on read, ÷1000 on set). `get_geometry`/`apply_geometry`/`manual_seed`/prefs `_assemble` still emit µm; the config key stays `lsd_um` (µm); calibration-file writers (`paramstest.txt`, `calibration.json`) are unchanged (always µm). Batch drift-status label → mm. **Files:** `tab_view.py`, `tab_calibrate.py`, `prefs_dialog.py`, `tab_batch.py`, docs. **Roll back:** `git revert c4e1c12` to return every Lsd field to a µm display. Purely a display-layer change — no stored/file/calculation values are affected either way.

`d6df1f8` — Fix "Populating font family aliases" startup warning (2026-07-13)

Effect: Removed the `qt.qpa.fonts: Populating font family aliases` warning (and its ~40 ms startup cost) that Qt emitted because the code named the nonexistent family "Monospace" — both via `QFont("Monospace", …)` and the CSS generic `font-family:monospace`. Now names real per-platform families (Menlo/Consolas/DejaVu Sans Mono/Courier New) via `style.MONO_FAMILIES` / `MONO_CSS` and `widgets._mono_font()` (QFont `setFamilies` + Monospace style hint). **Files:** `style.py`, `widgets.py`, `tab_export.py`, `tab_view.py`. **Roll back:** `git revert d6df1f8` (cosmetic; only affects fixed-width font selection and re-introduces the harmless warning).

`455ca4e` — Calibrate: multi-column results readout + Send-to-Data-Viewer (2026-07-13)

Effect: (1) The Calibrate **Results** tab shows the full parameter set exactly as written to `paramstest.txt` (Lsd, BC, tx/ty/tz, p0–p14, Parallax, Wavelength, px, NrPixelsY/Z, RhoD, SpaceGroup, LatticeConstant) in a 3-column, column-major grid (`_paramstest_pairs` via the shared writer + `_populate_param_grid`), with a strain/timing line and the named-distortion table below. (2) New → **Send to Data Viewer** button (`sendGeometryToViewer` signal + `_send_to_viewer`) pushes the calibrated λ/pixel/Lsd/beam-centre into the Data Viewer through new `DataViewerTab.set_geometry` (µm internal, Lsd shown mm, auto-BC off); wired in `app.py` as the reverse of the Viewer→Calibrate hand-off. **Files:** `tab_calibrate.py`, `tab_view.py`, `app.py`, docs. **Roll back:** `git revert 455ca4e`. Self-contained; depends on the mm-display change (`c4e1c12`) for the Viewer Lsd conversion.

`7e157e0` — Ship test_data; hide four optional tabs by default (2026-07-14)

Effect: (1) `test_data/` sample data (calibrant_ceria.tif/h5, nickel_stack.h5, nickel_tifs/, make_test_data.py) is now committed so the GUI's default paths work on a fresh clone — `.gitignore` re-includes it past the global `*.tif`/`*.h5` ignores via trailing negations, while `test_data/output/`, `out_temp.txt` and `.DS_Store` stay ignored. (2) `DEFAULT_VISIBLE_TABS = ["Calib. Refinement", "Pump Probe"]` — Corrections, PDF Analysis, Texture and Results & Export ship hidden (enable in Preferences

‣ Tabs). **Files:** `.gitignore`, `test_data/**` (added), `constants.py`, `config.example.json`, `tests/test_smoke.py`, docs. **Roll back:** `git revert 7e157e0` restores the all-optional-tabs default and removes the shipped data + re-ignores `test_data/`. Note: a per-user config with its own `ui.visible_tabs` overrides the shipped default regardless.

`6332b3d` — Fix launch crash on fresh Linux/Windows (colormap w/o matplotlib) (2026-07-14)

Effect: Fixes the GUI failing to start on a clean env where every always-on tab's image viewer crashed with `'NoneType' object has no attribute 'getColors'`. Cause: `DEFAULT_COLORMAP` "hot" (and the whole `COLORMAPS` list) are matplotlib colormaps; `pg.colormap.get("hot")` returns None without matplotlib, and that None was passed into pyqtgraph. New `widgets._resolve_cmap()` resolves native → matplotlib → native fallback → grayscale ramp and never returns None (used by ImageViewer, WaterfallViewer, Texture); matplotlib added as a declared dependency (`pyproject` + `environment.yml` matplotlib-base) so the named maps actually resolve. The downstream "Geometry hand-off wiring failed / QWidget has no attribute pushGeometry" log lines were only a symptom of the tabs becoming placeholders and disappear with this fix. **Files:** `widgets.py`, `tab_texture.py`, `pyproject.toml`, `environment.yml`, `tests/test_smoke.py`. **Roll back:** `git revert 6332b3d` (reintroduces the crash on matplotlib-less envs).

`72a1770` — env: drop midas_suite (unblocks `conda env create`) (2026-07-14)

Effect: `conda env create -f environment.yml` was aborting because the pip section's `midas_suite` meta-package pulls `midas-index 0.7.3`, whose sdist fails to build against `scikit-build-core>=0.8` (`Use cmake.version instead of cmake.minimum-version`). The GUI never uses midas_suite's extras; the backends it actually imports (midas-calibrate-v2/-integrate-v2/-calibrate/-hkls/-distortion) are declared in `pyproject.toml` and pulled by the editable `-e .` install. Removed the redundant `midas_suite` line; README updated. (Unrelated to the repo: the `libarchive.19.dylib` / conda-libmamba-solver errors in that log are a broken base-Anaconda mamba, worked around with `--solver=classic`.) **Files:** `environment.yml`, `README.md`. **Roll back:** `git revert 72a1770` (re-adds midas_suite and its build failure).

`68313f8` — Pin environment.yml to a verified NumPy-1 set + README recreate steps (2026-07-14)

Effect: Makes `conda env create` reproduce a known-good environment. Fresh installs previously pulled the newest backends (numba 0.66 → NumPy 2.x) against conda torch 2.2.2, which can't interop with NumPy 2 (`torch.from_numpy: Numpy is not available`). Pinned every package to the versions from the user's working base Anaconda — the **NumPy-1 family**: numpy=1.26.4, scipy=1.13.1, h5py=3.11.0, tifffile=2023.4.12, pyqtgraph=0.14.0, matplotlib-base=3.8.4 (conda); PyQt5==5.15.10, torch==2.4.0, numba==0.59.1, and the NumPy-1-era MIDAS backends (calibrate-v2 0.3.3, integrate-v2 0.1.0, calibrate

0.2.3, hkls 0.4.1, distortion 0.2.0) via pip; then `-e .`. Dropped the conda pytorch/cpuonly lines + pytorch channel (torch now via pip). `pyproject.toml`: lowered `midas-calibrate` floor 0.2.7 → 0.2.3 so the proven base version satisfies `-e .`. README gains a "Recreating the environment" section (remove + create), a conda-solver/libarchive note, and the CPU-only torch install note. Verified end-to-end: fresh env resolves clean; GUI + a real nickel-frame integration run (numpy 1.26.4 + torch 2.4.0). **Files:** `environment.yml`, `pyproject.toml`, `README.md`. **Roll back:** `git revert 68313f8` (returns to ranged deps; a fresh env would again risk the NumPy-2/torch mismatch). To move forward to a NumPy-2 stack instead, bump torch>=2.3 and the backends to their numba-0.66 releases together.

`3a6ecb9` — Calibrate Results: all-text multi-column readout (drop distortion table) (2026-07-14)

Effect: The Calibrate **Results** tab is now entirely text. Removed the distortion `QTableWidget`; the distortion coefficients appear in the same multi-column parameter grid, with the paramstest `p0–p14` slots relabelled to their names (iso_R2, a1, phi1, …) via `helpers._PARAMSTEST_DISTORTION`. Larger font (12 pt) and roomier row/column spacing for readability. Removed the `DistortionTable` import + `set_distortion` call. **Files:** `tab_calibrate.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 3a6ecb9` (restores the named-distortion table widget below the grid). `DistortionTable` still exists in `widgets.py` (used only here), so a revert is clean.

`b1642fa` — Fix calibration TypeError (initial_BC_y) + add scikit-image (2026-07-14)

Effect: Fixes calibration failing with `calibrate() got an unexpected keyword argument 'initial_BC_y'` on the pinned (base-matching) midas-calibrate-v2 0.3.3, whose `calibrate()` has no beam-centre seed parameter (only newer backends add it). New `calib._supported_kwargs(fn, kwargs)` filters kwargs to the installed callable's signature (keeps `initial_Lsd`, drops the unsupported `initial_BC_y/z`, logs it), so the GUI tolerates backend-version signature drift. Also pins **scikit-image=0.23.2** in `environment.yml` — midas-calibrate-v2's `auto_seed_calibrant` needs it; without it, seeding degrades to the coarse arc fallback. Verified: manual-seed one_shot calibration of the shipped ceria runs end-to-end (numpy 1.26.4 + torch 2.4.0). **Files:** `calib.py`, `environment.yml`. **Roll back:** `git revert b1642fa` (reintroduces the TypeError on backends lacking the BC-seed args, and removes the scikit-image pin).

`df544d2` — Data Viewer: tilt/distortion-aware radial integration (2026-07-14)

Effect: When a loaded calibration file carries the full geometry (tilts + distortion), the Data Viewer's radial integration goes through the MIDAS engine (`build_integration_context` + `integrate_frame`) instead of concentric-circle binning, so pixels map through the calibrated tilts/distortion. `_load_calibration` now also reads the full geometry via `geometry_fields_from_file` (falls back to scalar/circle mode if absent) and shows the active mode. `_midas_radial` caches the binning geometry per (geometry, R-bin, image shape, static mask) and reuses it across frames (fast `hard` kernel; loader's static composite mask so the cache holds); `_radial_integrate` branches to it with a guarded fallback to circle binning on error. **Files:**

`tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert df544d2` (reverts to circle-only radial integration in the Data Viewer). Self-contained; the MIDAS-engine primitives it reuses are unchanged.

`bc86d74` — Data Viewer: Top-N pixels + mask-aware stats + full-geometry receive (2026-07-19)

Effect: Adds a "Top-N pixels" toolbar toggle that marks the N brightest pixels (crosshair + circle) and feeds their values to the intensity-statistics panel, re-ranking live on frame/mask changes. Masked pixels (mask file + intensity-range mask) are excluded from the Top-N ranking and its stats/histogram. `_midas_radial` now unions the static mask with the per-frame intensity-range mask and fingerprints the mask by content so the cached binning context rebuilds when the mask changes. The tab also accepts a *full* geometry dict (tilts + distortion + detector size) from Calibrate and routes integration through the MIDAS engine when it arrives. The stats panel readout box auto-sizes to its content (no inner scrollbar) with the histogram as the single flexible child; `DataLoaderPanel` is now a `QWidget` hosting a scroll area + stats panel in a draggable vertical splitter. **Files:** `tab_view.py`, `widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert bc86d74`.

`b0a5cc3` — Calibrate: per-coefficient distortion, frame averaging, seed feedback (2026-07-19)

Effect: Adds `DistortionRefineDialog` (behind the "…" next to the Distortion checkbox) to pick any of the 15 harmonics or use η-fold presets; `calib.py` maps each selected v2 harmonic to its v1 p-slot (via `DISTORTION_ISO` / `DISTORTION_PRESETS` in constants). Adds an "Average frames" card (start:end:skip, streamed via `DataLoaderPanel.average_frames`), seed tilts (tx/ty/tz) carried into the v1 params, a "Feed result back to seed" option, and a "→ Send to Data Viewer" that now sends the full geometry (tilts + distortion + detector size). Drops the 310px bottom-panel cap and adds a non-collapsible splitter. **Files:** `calib.py`, `tab_calibrate.py`, `dialogs.py`, `constants.py`, `widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert b0a5cc3`.

`b6f1681` — Batch: read-only "Calibration values" card (2026-07-19)

Effect: Adds a Calibration values card to the Batch parameter column showing the geometry actually driving integration (λ, Lsd, BC, tilts, pixel sizes, detector size, non-zero distortion count), resolved from the live Tab-2 result or a calibration file and refreshed on source/path change. Also lets the Log panel fill its splitter space. **Files:** `tab_batch.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert b6f1681`.

`49623ae` — Pump Probe: publication plotting, delay colour-bar, default mask (2026-07-19)

Effect: Publication-quality plot defaults (clean 2.0px lines, 13pt fonts, dashed grid, refactored `_rainbow_cmap`); a continuous delay→colour bar (`GradientLegend`) replaces the many-row legend past a threshold, with left/right legend anchoring; a log-delay x-axis option for kinetics; and a default detector mask wired via new `DataLoaderPanel.add_mask_file` / `MaskSelector.add_file_source`. TR-XRD defaults now point at a local-only `test_data_pump_probe/` folder kept out of git. **Files:** `tab_pumpprobe.py`, `constants.py`, `widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 49623ae`.

`b251ca8` — chore: gitignore context, internal docs, large sample sets (2026-07-19)

Effect: Ignores the local `.context/` scaffold and `CLAUDE.md`, the internal-only PDF calculation write-up, and the large local sample dirs (`test_data/17BM/`, `test_data/test_s25ide/`) via directory-level ignores that override the `test_data/**` re-includes. **Files:** `.gitignore`. **Roll back:** `git revert b251ca8`.

`1769f69` — Data Viewer: live PV stream (Live Data card) (2026-07-20)

Effect: Subscribes to an EPICS PVA image PV via `pvapy` and pushes frames straight through the existing image/corrections/ring/radial-integration pipeline — no separate viewer window. “Live Data” is a collapsible card (own title-bar checkbox) above the Data card; unchecking stops an active stream. Data card gets a small ⟳ reload button to restore the static file/folder/HDF5 source after stopping a stream. `pvapy==5.4.1` is now a required, pinned dependency (chosen over latest to avoid upgrading the numpy/pyqtgraph pins). `MainWindow` calls a generic `tab.shutdown()` hook on close so a live stream stops cleanly on app exit. **Files:** `widgets.py`, `tab_view.py`, `app.py`, `pyproject.toml`, `environment.yml`, `documentation/gui_documentation.md`, `tests/test_live_stream.py`. **Roll back:** `git revert 1769f69`.

`0b4326d` — Fix: cap native thread pools to prevent calibration crash (2026-07-22)

Effect: Clicking “Run Calibration” crashed the app outright — a native, uncatchable `Bus error`, not a Python exception. Root cause: `CalibrationWorker` (a `QThread`) triggers a multi-threaded `numpy.linalg.inv()` deep inside `midas-calibrate-v2`’s HKL/ring generation (likely surfaced by the recent 0.3.3 → 0.5.2 bump); running from a `QThread` while the Qt event loop is alive, that races against PyTorch’s own OpenMP pool and corrupts memory. Reproduced deterministically at two call sites (`numpy.linalg.inv` during ring generation, `scipy.ndimage.median_filter` during seeding); both disappear once native thread pools are capped to 1. Every heavy operation in this app runs inside a `QThread` (17 worker classes in `workers.py`), so the fix caps `OPENBLAS_NUM_THREADS` / `OMP_NUM_THREADS` / `MKL_NUM_THREADS` / `VECLIB_MAXIMUM_THREADS` / `NUMEXPR_NUM_THREADS` to 1 process-wide via `os.environ.setdefault` in `_paths.py` — already home to the sibling `KMP_DUPLICATE_LIB_OK` workaround for the same underlying duplicate-OpenMP-runtime issue — rather than patching each worker individually. Verified: offscreen run of the full calibration pipeline to convergence with no crash;

`pytest` 15/15 green. **Files:** `midas_gui/_paths.py`. **Roll back:** `git revert 0b4326d` (restores multi-threaded BLAS; calibration/other workers become exposed to this native-crash class again on macOS).

`234ded9` — Live viewer: preserve manual colormap/level changes across frames (2026-07-24)

Effect: `ImageViewer._redisplay` recomputed vmin%/vmax% percentile levels on every incoming frame (live or otherwise), silently overwriting a level range the user had dragged on the pyqtgraph histogram mid-acquisition. Now tracks manual histogram edits via `sigLevelsChanged` (guarded against feedback from our own programmatic `setImage` calls with a suspend flag) and keeps pinning to the last manual levels until the user explicitly changes the vmin%/vmax% spin boxes, which resets back to auto. **Files:** `widgets.py`. **Roll back:** `git revert 234ded9`.

`69f470c` — Data Viewer: widen the image/radial-plot splitter handle (2026-07-24)

Effect: The vertical splitter between the image view and the radial-integration panel had no explicit handle width (unlike the outer horizontal splitter), making it thin and hard to grab. Set `setHandleWidth(8)` to match. **Files:** `tab_view.py`. **Roll back:** `git revert 69f470c`.

`96f8add` — Radial profile: stop forcing Y-axis min to -1000 on every update (2026-07-24)

Effect: `ProfileViewer._replot` unconditionally called `setYRange()` with a hardcoded -1000 floor on every redraw, clobbering any manual Y-range the user set. The auto lower bound is now `0.9 * current data minimum`; once the user manually changes the Y range (drag, wheel-zoom, or the axis context menu's numeric entry — detected via `sigYRangeChanged` with the same suspend-flag pattern as `234ded9`), that value is kept for the rest of the session instead of being recomputed. Also dropped the hardcoded `setLimits(yMin=−1000/1)` hard floors. **Files:** `widgets.py`. **Roll back:** `git revert 96f8add`.

`3eb4a44` — Calibrate: recolor Pick-Ring fit overlay to blue (2026-07-24)

Effect: `PickableImageViewer`'s Pick-Ring points, fitted circle, and fitted-center marker used the same amber (`#f0c060`) as the Data Viewer's Simulate-rings overlay, making the two indistinguishable when both were visible on the same image. Pick-Ring's fit overlay is now blue (`#2a7fd4`); the separate Pick-BC "+" click marker (already `#00aaff`) and Simulate-rings amber were left unchanged. **Files:** `widgets.py`. **Roll back:** `git revert 3eb4a44`.

`ecf6d01` — Data Viewer: make Simulate rings a live toggle instead of one-shot (2026-07-24)

Effect: "Simulate rings" is now a checkable toggle rather than a single-click action. While on, it resimulates automatically whenever material, lattice constants, space group, or geometry (wavelength/Lsd/pixel size/max 2θ) change — mirroring the existing `_rad_auto`-style "recompute on change" idiom already used for radial integration. Beam-centre edits still just reposition the existing rings (unchanged; ring radii don't depend on BC), so no wiring was added there. **Files:** `tab_view.py`. **Roll back:** `git revert ecf6d01`.

`aa82cb6` — Preferences: add Devices tab; Data Viewer Live PV becomes a device dropdown (2026-07-24)

Effect: New **Preferences ▸ Devices** tab (add/remove/edit rows of name + prefix + PVA suffix), following the existing Materials/Calibrants table-tab pattern; persisted as a new `"devices"` list in the JSON config (replaces wholesale when present, same as materials/calibrants). Ships pre-filled with the 20-ID-D detectors extracted from the beamline's `make_det(...)` blueprint — `oryx20idd` (`20iddOR1:`), `s20idPil` (`20idPil`), `pg4` (`1idPG4:`), `gh1s` (`20idGH1s:`), `s20varex1` (`20IDFF:`) — all with PVA suffix `Pva1:Image`. The Data Viewer's Live Data "Live PV" field is now an editable combo box (`_NoScrollComboBox`) populated from this list: picking a device by name fills in its full PV (`prefix + PVA suffix`); typing any other PV by hand still works unchanged, with the same example placeholder text. **Files:** `constants.py`, `prefs_dialog.py`, `widgets.py`, `tests/test_live_stream.py`. **Roll back:** `git revert aa82cb6` (Devices tab and the Live PV dropdown both go; the Live PV field reverts to a plain text box).

`c156c62` — Fix image-view autorange drift; bound pan/zoom to image (2026-07-24)

Effect: Two bugs in the shared `ImageViewer` (Data Viewer / Mask Builder / Calibrate all use it): (1) the crosshair `InfiniteLine`s were added to the pyqtgraph `ViewBox` without `ignoreBounds=True`, so their position — which follows the mouse on every hover event — fed the auto-range bounds calculation. Once the bottom-left **A** (auto-range) button enabled *continuous* auto-range, the view kept re-fitting itself to wherever the cursor was, and only a right-click ▸ View All (which disables continuous auto-range as a side effect) made it static again. Fixed by excluding the crosshairs from bounds via `ignoreBounds=True`. (2) Pan/zoom had no limits, so scroll/drag could wander arbitrarily far from the image or zoom out indefinitely into empty space. Added `ViewBox.setLimits()` (computed from the image shape in `set_image()`): pan is bounded to roughly half an image-width/height of margin past each edge, and min/max zoom range are tied to the image size. **Files:** `widgets.py` (`ImageViewer.set_image`, new `_apply_view_limits`). **Roll back:** `git revert c156c62` (crosshair can drift the view again under continuous auto-range; pan/zoom become unbounded again).

`cde4bb2` — Radial profile plot: bound pan/zoom to the data extent (2026-07-24)

Effect: Same class of issue as `c156c62`, applied to the shared `ProfileViewer` (the radial-integration plot on the Data Viewer and Calibrate tabs): it had no `ViewBox` limits, so the plot could be scrolled/dragged arbitrarily far from the actual profile. `_replot()` now calls a new `_apply_view_limits(xmin, xmax, ymin, ymax)` on every redraw — computed from the current profile's X range (in whichever unit is selected: R (px) / 2θ / Q) and Y range (respecting the existing sticky-manual-Y-min behavior) plus a margin (15% X, 25% Y), via `ViewBox.setLimits(...)`. Recomputing per-replot means the bound tracks new profiles and axis-unit switches automatically. **Files:** `widgets.py` (`ProfileViewer._replot`, new `ProfileViewer._apply_view_limits`). **Roll back:** `git revert cde4bb2` (radial profile pan/zoom becomes unbounded again).

`5b8e3b3` — User profiles, full GUI state save/load, Calibrate tilt-ring rework (2026-07-27)

Effect: Three pieces of work landed in one commit: 1. **User profiles.** `settings.py` now keeps one config file per named profile under `<config dir>/profiles/<name>.json` (`profile_meta.json` tracks which is active), transparently migrating an existing single `config.json` into a profile named "Default" the first time it runs — no data lost, no explicit migration step. `load_config()` / `save_user_config()` / `reset_user_config()` / `user_config_path()` keep their existing signatures and now resolve to the active profile, so every existing call site (`constants.py`, `prefs_dialog.py`, `app.py`) needed no changes. New `constants.reload_from_config()` resets every `DEFAULT_*` global to its shipped value then re-applies the active profile's config on top (a plain re-`_apply` alone would leave a previous profile's override in place if the new profile doesn't mention that key). Preferences gets a **Profile row** — combo + New…/Duplicate…/Rename…/Delete — wired to this API. 2. **Full GUI state save/load.** New **File** menu, **Save GUI State…** (`Ctrl+S`) / **Load GUI State…** (`Ctrl+O`), dumps/restores every tab's fields to one JSON file via new `widgets_to_dict()` / `apply_dict_to_widgets()` dispatch helpers (`helpers.py`) and a `get_state()` / `set_state()` pair added to all 10 tabs plus the shared `DataLoaderPanel` / `FieldSelector` / `MaskSelector` / `CorrectionFlagsWidget` widgets. Loading a state re-triggers each tab's own file-loading for path-backed fields (images, masks, dark/bright/background, HDF5 datasets) but deliberately does **not** re-run long pipelines (Fit, Batch Integrate, PDF transform, Refinement) — those need one manual click of the tab's own action button afterward. `MaskTab` / `CalibrationTab` additionally write small sidecar files (`<stem>_mask.tif`, `<stem>_calibration.json`) next to the state file so an in-progress mask or fit result that hasn't been exported anywhere else isn't silently lost. 3. **Calibrate tilt-ring rework** (previously uncommitted, folded in here): the "Corrected" ring overlay is now drawn synchronously from the fitted ty/tz tilt (`_draw_corrected_rings`) instead of a background `CorrectedRingsWorker` forward-model thread (removed from `workers.py`). Data Viewer gained matching `ty` / `tz` tilt fields for its ring simulation, and every Ring-simulation spin-box's step size (λ, max 2θ, Lsd, pixel, BC, tilt) is now configurable via Preferences ▸ Data Viewer / the `viewer_steps` config key instead of hardcoded. **Files:** `settings.py`, `constants.py`, `prefs_dialog.py`, `helpers.py`, `widgets.py`, `app.py`, `tab_view.py`, `tab_calibrate.py`, `tab_mask.py`, `tab_batch.py`, `tab_refine.py`, `tab_pumpprobe.py`, `tab_corrections.py`, `tab_pdf.py`, `tab_texture.py`, `tab_export.py`, `workers.py`, `tests/test_config.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 5b8e3b3` — reverts all three pieces together (they share edits in `tab_view.py` / `tab_calibrate.py`, so a partial revert needs a hand-picked patch, not a plain `git checkout <path>`). Existing single-profile configs are unaffected either way since `profiles/Default.json` is a copy, not a move, of the original `config.json`.

`e14c1ea` — Data Viewer: tilt-aware profile, ring fixes, calibration export, plot zoom persistence (2026-07-28)

Effect: Seven requested fixes/features plus two follow-up plot bugs found while testing them, all in the Data Viewer tab: 1. **Tilt-corrected radial profile without a calibration file.** New `DataViewerTab._effective_calib_geom()` returns `self._calib_geom` if a calibration is loaded, otherwise synthesizes a geometry from the Ring-simulation card's own `ty`/`tz`/BC/Lsd/pixel fields whenever they're non-zero. `_radial_integrate()`/`_midas_radial()` route through this instead of only checking `_calib_geom`, so the profile's R/2θ/Q axis stays tilt-consistent with the already-correct 2D ring overlay even before any calibration file is loaded. `_midas_radial`'s integration-context cache key changed from `id(g)` to a value tuple (the synthesized geometry is a fresh dict each call, so `id()` never hit the cache). 2. **Ring 2θ-range cutoff bug.** `_redraw_rings` capped ring radius at the image's pixel diagonal (`math.hypot(NY, NZ)`), silently dropping any ring past that regardless of the configured "max 2θ" — for far-detector/ fine-pixel geometries this fell around 35-40°. Replaced with a bare `rad > 0 and math.isfinite(rad)` guard; pyqtgraph's ViewBox already clips anything drawn outside the visible image. 3. **Configurable ring thickness.** New `DEFAULT_RING_WIDTH = 2.0` constant and a spin box in the Ring-simulation card, wired into `_redraw_rings`'s pen width and into `_state_widgets()` (round-trips through Save/Load GUI State). 4. **"Simulate rings" turns green while live.** New `QPushButton#primary:checked` rule in `style.py` — confirmed via grep that the Data Viewer's live-toggle button is the only checkable `primary_btn` in the app, so this can't affect any other tab. 5. **Save calibration from the Data Viewer.** Three new buttons (Save JSON / Save params (.txt) / Save PONI) export whichever geometry is currently in effect (`_export_geom()`, mirroring `_effective_calib_geom()`). JSON uses the same bare-key shape `geometry_fields_from_file()` reads back; `.txt` reuses the existing `write_standalone_paramstest()`; `.poni` uses a new `helpers.write_poni()` that inverts the existing PONI reader's convention (`Poni1/2 = BC_y/z * pxY/Z`, `Distance = Lsd`, SI units) — tx/ty/tz tilts have no PONI Rot1-3 equivalent and are documented as dropped on export, matching the reader's existing documented limitation. 6. **`Ctrl+S` overwrites a loaded/saved GUI-state file.** `MainWindow` tracks `_gui_state_path`, set on every successful save or load; a plain `Ctrl+S` now writes straight to that path with no dialog once one exists. New `Ctrl+Shift+S` ("Save GUI State As…") always prompts, and becomes the new target for subsequent plain saves. 7. **Radial-profile X-axis floors at 0.** Both `ProfileViewer._replot`'s `setXRange` call and `_apply_view_limits`'s pan/zoom `xMin` bound now clamp to `max(0.0, ...)`.

Testing these surfaced two more bugs in `ProfileViewer`, fixed in the same commit: - **Y-range could get permanently stuck.** The old `_manual_ymin` latch recorded *any* Y-range-changed signal, including ones pyqtgraph fires internally (e.g. autorange during `curve.setData()`) well before the method's own suspend-flag window started — once latched, every future replot used that stale value regardless of new data. Fixed by wrapping the entire `_replot()` body (curve/band updates, limit-setting, everything) in one suspend flag, so only genuine mouse-driven pan/zoom is ever recorded. - **Zoom reset on every parameter change.** Replaced the unconditional `setXRange` / `setYRange` calls with tracked `_user_xrange` / `_user_yrange` : the plot still auto-fits fresh data when the user hasn't manually zoomed, but a manual zoom/pan is now restored after every replot instead of being wiped by the next profile update. The remembered range is only dropped when it would be meaningless in the new context — switching the R/2θ/Q axis unit clears the X range, toggling Log Y clears the Y range. **Files:** `midas_gui/app.py` , `midas_gui/constants.py` , `midas_gui/helpers.py` , `midas_gui/style.py` , `midas_gui/tab_view.py` , `midas_gui/widgets.py` , `documentation/gui_documentation.md` . **Roll back:** `git revert e14c1ea` . All pieces are additive/isolated (new widgets, a new geometry-resolution helper, a new writer function, and a `ProfileViewer` range-tracking rewrite) — no other commit builds on this one.

`3c15ae7` — Data Viewer: native-menu-backed manual axis limits, Top-N intensity floor (2026-07-29)

Effect: Reworks the radial-integration plot's and intensity-histogram's Auto/Manual toggle to defer axis-value entry to pyqtgraph's own existing right-click axis "Manual" min/max fields, instead of a separate custom field row (an earlier same-day attempt at this feature used custom fields and was corrected): 1. New `_add_auto_manual_buttons(plot_widget, on_auto, on_manual)` in `widgets.py` hides pyqtgraph's native auto-range corner button and replaces it with a small "A"/"M" `QPushButton` pair in the same spot, wired to `clicked` (not `toggled`) so that **reclicking the already-active button** still fires its handler — "A" re-fits immediately to current data, "M" snaps back to the held manual range, discarding any pan/zoom drift in either mode. 2. New `_install_manual_axis_capture(plot_widget, callback)` hooks each axis's native `ViewBoxMenu` `minText` / `maxText` `editingFinished` signals (pyqtgraph already applies the typed value to the view itself) and mirrors the committed range into a `self._manual_range` tuple tracked independently of `ViewBox.state['targetRange']` (which changes on drag/ pan too, not just explicit edits — needed for reclick-to-reset to mean something different from "wherever the mouse left it"). 3. `ProfileViewer` / `IntensityStatsPanel` `_replot` / `_redraw_hist` skip their auto-fit/clamp logic entirely while `_manual_mode` is set and call the new `_apply_manual_range()` instead (clears any `setLimits()` pan/zoom bound, then `setXRange` / `setYRange` with `padding=0` from the held tuple) — this is what makes an exact typed value (e.g. `0`) survive every live- acquisition redraw instead of being clamped/padded by that frame's own `_apply_view_limits()` recompute. 4. Top-N pixel marking (`DataViewerTab._show_topn` , `tab_view.py`) gains an "I >" checkbox + threshold field next to the N spin box, matching the existing `_imask_on` / `_fspin` convention; pixels at/below the threshold are excluded from ranking before `argpartition` , so fewer than N (or zero) markers show when fewer pixels clear the floor. 5. Unrelated: `constants.DEFAULT_DEVICES` PVA device name/prefix updates (`oryx20idd` → `20iddNF` , `20iddOR1:` → `20idOR1:` , `20idPil` → `20idPil:` , `gh1s` → `20iddTomo` , `s20varex1` → `20iddFF`), bundled into this commit at the user's request. **Files:** `midas_gui/constants.py` , `midas_gui/tab_view.py` , `midas_gui/widgets.py` , `documentation/gui_documentation.md` . **Roll back:** `git revert 3c15ae7` . Self-contained — no later commit depends on the new helpers or the Top-N threshold field.

`53f8f19` — Widen non-data-derived numeric caps across tabs (2026-07-29)

Effect: Several `QSpinBox` / `QDoubleSpinBox` range caps had no physical or data-derived justification (arbitrary round numbers picked when the field was first added) and were clamping legitimate inputs; raised them: 1. Data Viewer's simulated-ring `ty` / `tz` tilt fields and Calibrate's seed `tx` / `ty` / `tz` fields: `±10°` → `±180°`, matching the full range other angle fields in the app already allow. 2. Data Viewer max-2θ: `1–90°` → `0.001–180°`. 3. Calibrate E-M/LM iteration counts and multi-panel geometry (panel counts, size, gap): capped at 20/2000/50/10000/1000 → `1_000_000`. 4. Corrections empty-frame/Compton scale, absorption μR, gain-training steps/ lr/unity-weight/smooth-weight: capped at 1–20 → effectively unbounded (`1e6` – `1e9`). 5. Mask hot/dead factor, frozen-fraction, stride, and learnable-mask steps/ lr/sparsity: capped at 1–2000 → effectively unbounded. 6. Refine optimizer lr and iteration count: capped at 10/2000 → effectively unbounded. 7. Batch drift `n_knots`: capped at 20 → `1_000_000`. **Files:** `midas_gui/tab_batch.py`, `midas_gui/tab_calibrate.py`, `midas_gui/tab_corrections.py`, `midas_gui/tab_mask.py`, `midas_gui/tab_refine.py`, `midas_gui/tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 53f8f19`. Self-contained — no later commit depends on the widened ranges.

`05ce224` — Data Viewer: mask enable checkboxes, calibration card reposition + ty/tz sync, intensity-range title checkbox (2026-07-30)

Effect: Three independent Data Viewer tweaks: 1. `MaskSelector` (`widgets.py`) rows gain a checkbox to include/exclude a mask from the composite without deleting it; `composite_mask()` now OR's only *checked* sources. The "Tab 1 mask" row (auto-populated from the Mask Builder tab) is always inserted/kept at the top of the list. `get_state()` / `set_state()` persist each source's `enabled` flag. 2. The Load calibration card moved below the Ring simulation card's Simulate button, and loading a calibration file now also fills the Ring-simulation card's ty/tz tilt fields (previously only λ/Lsd/px/BC). 3. The Intensity range card's title bar is now the on/off checkbox itself ("Exclude out-of-range pixels") instead of a separate checkbox line inside the card. **Files:** `midas_gui/tab_view.py`, `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 05ce224`. Self-contained — no later commit depends on the mask `enabled` flag, the card reposition, or the title-bar checkbox.

`0e9ed21` — Data Viewer: support multiple ring-simulation materials at once (2026-07-29)

Effect: Ring simulation card holds a list of materials instead of one, so a sample phase can be overlaid on a calibrant (or any N materials) at the same time: 1. New `MaterialDialog` (factored out of the old inline lattice fields) edits one material's name/preset/lattice/space-group/cubic-checkbox. Opened by clicking a material row's underlined name button. 2. Each material row is a checkbox (show/hide that material's rings), a clickable color swatch (`QColorDialog`, drives that material's ring lines + hkl labels on both the image overlay and radial-profile plot), the clickable name, and a ✕ delete button (disabled when it's the last remaining row). **+ Add material** appends a new row with the next color from a 10-entry cycling palette (`_MATERIAL_COLORS`); a single default **Ni (FCC)** row is seeded at startup, matching prior behavior. 3. Geometry fields (λ, max 2θ, Lsd, pixel size, beam centre) stay shared across all materials — only lattice/SG/color/visibility are per-material. 4. `_simulate()` now loops materials, simulating rings per enabled one and collecting per-material errors instead of aborting on the first exception; `_redraw_rings()` / `_refresh_profile_markers()` draw each material's rings in its own color. 5. `ProfileViewer.set_ring_markers()` (`widgets.py`) takes a list of `{"radii": [...], "color": "#hex"}` groups instead of a flat radii list, so the radial-profile plot can show multiple colored ring sets; `tab_calibrate.py` updated to pass its single ring set as a one-entry group list. 6. `DataViewerTab.get_state()` / `set_state()` persist/restore the materials list (replacing the old single-material `mat` / `a..sg` / `cubic` state fields) so GUI-state save/load round-trips multi-material setups. **Files:** `midas_gui/tab_view.py` , `midas_gui/tab_calibrate.py` , `midas_gui/widgets.py` , `documentation/gui_documentation.md` . **Roll back:** `git revert 0e9ed21` . Self-contained — no later commit depends on the multi-material materials list or the `set_ring_markers()` groups signature.

`911f8ac` — Data Viewer: Live Data "Use Buffer" ring buffer for stack analysis on live streams (2026-07-30)

Effect: Live Data card gains a **Use Buffer** toggle + **N** spin box (`DataLoaderPanel` , `widgets.py`) that captures the last N live frames into an in-memory `deque` ring buffer, so Projection and every other stack-based analysis become usable on live data instead of only on loaded HDF5/folder stacks: 1. Clicking **Use Buffer** arms it: turns yellow (`Buffering… (n/N)`) and appends each incoming live frame to the buffer as it streams. 2. When no new frame arrives for ~2 s (streaming paused, or **Stop** clicked), a single-shot `QTimer` (`_buffer_stall_timer`) freezes the buffer into a navigable stack: `_nframes` / `_setup_navigator()` update so the frame slider/spin/prev-next and **Project stack** work exactly as they would on a loaded stack; the button turns green (`Buffer Ready (N)`). 3. `_get_frame()` / `full_stack()` read from the frozen buffer when active, otherwise fall through to the existing stack/paths/h5 sources unchanged. 4. Starting a new live stream or loading static data calls `_reset_buffer()` , discarding any existing buffer and turning the toggle off. **Files:** `midas_gui/widgets.py` , `documentation/gui_documentation.md` . **Roll back:** `git revert 911f8ac` . Self-contained — no later commit depends on the ring-buffer fields or `_get_frame()` / `full_stack()` buffer branch.

`a88ba1f` — Data Viewer: fix live ring-buffer race between GUI and worker threads (2026-07-30)

Effect: `_on_live_frame()` appends to the "Use Buffer" ring buffer (`DataLoaderPanel._buffer` , `widgets.py`) on the GUI thread while background `QThread` s (e.g. `ProjectionWorker.run()` , reached via `full_stack()`) read the same `deque` with no synchronization — a live frame arriving mid-read could raise "deque mutated during iteration" or hand analysis a torn frame set. Added a `threading.Lock` (`_buffer_lock`) guarding every read/write of `self._buffer` and the `_buffer_frozen` flag checked alongside it, in `_get_frame()` , `full_stack()` , `_on_live_frame()` , `_on_buffer_toggled()` , `_on_buffer_stalled()`, and `_reset_buffer()`. **Files:** `midas_gui/widgets.py`. **Roll back:** `git revert a88ba1f`. Self-contained — the lock only wraps existing buffer accesses, no signature/state-shape changes.

`2cfd9cd` — Data Viewer: fix refresh-timer starvation during fast live streaming (2026-07-30)

Effect: `dataChanged` was connected to `self._refresh_timer.start()` directly. `QTimer.start()` on an already-armed `setSingleShot(True)` timer restarts its countdown rather than queuing a fire, so a continuous burst of events faster than the 60ms interval (e.g. live frames streaming quickly) could defer the refresh indefinitely — the displayed image/stats/rings would appear frozen mid-burst even though data kept arriving. Added `_on_data_changed()`, connected to `dataChanged` in place of the old lambda, which only calls `._refresh_timer.start()` while the timer is idle (`not isActive()`), turning it into a throttle (fires at least once per interval) instead of a restart-on-every-event debounce that can starve. **Files:** `midas_gui/tab_view.py`. **Roll back:** `git revert 2cfd9cd`. Self-contained — restores the direct lambda connection.

`839770d` — Data Viewer: cap live buffer (“Use Buffer”) to 100 frames (2026-07-30)

Effect: The ring-buffer **N** spinbox (`widgets.py`) allowed up to 2000 frames, each stored as a full float32 array — for a large-format detector this could allocate tens of GB with no warning. Lowered `setRange(2, 2000)` to `setRange(2, 100)`; tooltip and `gui_documentation.md` updated to note the cap. **Files:** `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 839770d`. Self-contained — restores the old range only.

`08392c6` — Data Viewer: move “All frames” stats computation off the GUI thread (2026-07-30)

Effect: `_all_frame_values()` (reached from `_update_stats()` when the stats scope is “All frames”) read the entire stack/live buffer and applied field corrections synchronously on the GUI thread — unlike every other heavy operation in this file, which already runs in a background `QThread`. For a large HDF5/folder stack, or a full live ring buffer, this could visibly freeze the UI. 1. Added `AllFrameStatsWorker` (`workers.py`), following the same pattern as `ProjectionWorker`: GUI-derived inputs (dark/bright/background arrays, composite mask, intensity-range thresholds) are snapshotted on the GUI thread before the worker starts, so `run()` only touches numpy data. 2. `_update_stats_all_frames()` (`tab_view.py`, replaces `_all_frame_values()`) launches the worker and, if a new request arrives while one is already running, coalesces it into a single re-run afterward via `_stats_all_frames_dirty` instead of queuing redundant work. **Files:** `midas_gui/tab_view.py`, `midas_gui/workers.py`. **Roll back:** `git revert 08392c6`. Self-contained — no later commit depends on `AllFrameStatsWorker` or the removal of `_all_frame_values()`.

`57ced2f` — Data Viewer: debounce intensity-mask spinbox edits (2026-07-30)

Effect: The intensity-mask “pixel ≤” / “pixel >” spinboxes triggered `_update_stats()` / `_radial_integrate()` (and Top-N re-ranking, if active) on every `valueChanged` signal — i.e. once per keystroke or arrow-click — doing potentially expensive recomputation while the user was still mid-edit. 1. Added `_imask_debounce_timer`, a single-shot `QTimer` (120ms), alongside the existing `_refresh_timer` in `__init__`. 2. `_imask_lo` / `_imask_hi` now connect to a new `_on_imask_value_changed()`, which updates the (cheap) mask overlay immediately for visual feedback but only (re)starts the debounce timer for the heavier recompute; the timer’s `timeout` fires the existing `_on_imask_changed()` once edits settle. The on/off toggle (`_imask_on`) is unaffected — it still calls `_on_imask_changed()` directly. **Files:** `midas_gui/tab_view.py`. **Roll back:** `git revert 57ced2f`. Self-contained — restores the direct `valueChanged.connect(self._on_imask_changed)` wiring.

`81b8ea8` — Data Viewer: warn when composite mask drops a source (2026-07-30)

Effect: `MaskSelector.composite_mask()` OR’s every enabled mask source together, but silently skipped any source that failed to load or whose shape didn’t match the union built so far — a user could enable a mask file believing it was in effect while it was quietly excluded. 1. `composite_mask()` now collects the label (and, for shape mismatches, the offending shape) of every dropped source into `self._mask_warning`. 2. `_refresh()` shows the warning in the existing status label (amber `#e0a030`) alongside the normal source count; mutating the source list (add/remove/toggle/`set_tab1_mask`) clears the stale warning until the next `composite_mask()` call recomputes it. **Files:** `midas_gui/widgets.py`. **Roll back:** `git revert 81b8ea8`. Self-contained — restores the silent drop and the plain “N mask source(s)” status text.

`3d96cb1` — Data Viewer: add hardware-free Sim Detector for Live Data testing (2026-07-31)

Effect: Testing the Live Data card required a real EPICS PVA detector stream from beamline hardware — no way to exercise it standalone. 1. New `midas_gui/sim_detector.py` : `SimDetectorServer` runs a real `pvapy.PvaServer` publishing genuine NTNDArray frames (same plumbing a real detector’s PVA image plugin uses via `AdImageUtility`), so it’s indistinguishable from a real stream to `PvaLiveSource`. Defaults to an Eiger2 500K shape (1030×514 px, matching the repo’s existing `DEFAULT_PIXEL_UM` comment), counts in `[0, 60000]` , 5 Hz — all constructor parameters. `ensure_running()` / `stop_all()` give a process-wide registry keyed by channel name. 2. `constants.DEFAULT_DEVICES` gains a “Sim Detector” entry (PV `midasSim:Pva1:Image`); the config-overlay system only round-trips `name` / `prefix` / `pva_suffix` for devices, so the sim device is identified purely by its resolved PV string, not a custom marker key. 3. `DataLoaderPanel._start_live()` (`widgets.py`) lazily starts the sim server via `ensure_running()` the first time that PV is connected to. 4. `DataViewerTab.shutdown()` (`tab_view.py`) calls `sim_detector.stop_all()` alongside the existing `stop_live()`. **Files:** `midas_gui/sim_detector.py` (new), `midas_gui/constants.py`, `midas_gui/widgets.py`, `midas_gui/tab_view.py`. **Roll back:** `git revert 3d96cb1`. Self-contained — removes the sim device/file and its two call sites; no later commit depends on it.

`3b12fbf` — Image viewer: persist manual color-scale window across live frames (2026-07-31)

Effect: `234ded9` (2026-07-29) made a manually-dragged histogram LUT level range survive across incoming frames, but two gaps remained: the histogram's own zoom/pan window still reset on every redraw, and toggling Log/Linear reinterpreted a manual level's raw numbers in the new scale instead of converting them — both made a deliberately-set color window jump around unexpectedly. 1. `ImageViewer` gains `_manual_hist_range` (mirrors `_manual_levels` but for the histogram's own axis zoom) and `_suspend_hist_range_track`, tracked via a new `sigRangeChanged` connection and `_on_hist_range_changed()`. 2. `set_image()` gains a `reset_levels` flag: `True` (default) drops both manual windows and returns to the vmin%/vmax% percentile defaults; `False` — passed for a live-streaming frame update — leaves them in place. `DataLoaderPanel.is_live_frame_update()` reports whether the most recent `dataChanged` came from a live PVA frame, so `tab_view.py` and `tab_calibrate.py` can pass the right value. 3. `_redisplay()` now also re-zooms the histogram to the percentile window by default (`setHistogramRange(lo, hi, padding=0.1)`) so a single bad pixel can't stretch it into an unreadable sliver. 4. New `_on_log_toggled()` converts `_manual_levels` through log10/10**x when the Log checkbox flips (clamped to ±300 before exponentiating, since a manual level can sit far outside real data range and float `10**x` raises `OverflowError` rather than saturating), and drops `_manual_hist_range` so the view reframes tightly around the converted levels instead of carrying a now-mismatched zoom through the transform.** Files:** `midas_gui/widgets.py`, `midas_gui/tab_view.py`, `midas_gui/tab_calibrate.py`. **Roll back:** `git revert 3b12fbf`. Self-contained — restores the pre-`234ded9`-successor behavior (LUT levels still persist per `234ded9`, but histogram zoom resets and Log/Linear toggle no longer converts levels).

`f3a1b91` — Data Viewer: preserve armed "Use Buffer" state when Start is clicked (2026-07-31)

Effect: `_start_live()` unconditionally called `_reset_buffer()`, which discarded the buffer and flipped the **Use Buffer** button back to its off/gray state even if the user had already armed it (yellow `Buffering…`/green `Buffer Ready`) before clicking **Start**. That forced a redundant second click on **Use Buffer** after every Start. `_start_live()` now only calls `_reset_buffer()` when `_buffer_active` is `False`; when it's already `True`, it re-arms a fresh empty deque (same as `_on_buffer_toggled`'s checked branch) and restyles to `"filling"`, so buffering continues into the new stream without user intervention. **Files:** `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert f3a1b91`. Self-contained — restores the unconditional `_reset_buffer()` call on Start.

`2525277` — Data Viewer: unrestrict wavelength/Lsd/pixel-size ranges, tighten ring row (2026-07-31)

Effect: Ring simulation card's λ/Lsd/pixel-size spin boxes were capped at arbitrary limits (pixel size 1–5000 μm, λ 0.001–10 Å, Lsd 0.001–100000 mm); they now accept any positive value (λ 0.0001–1e6 Å, Lsd 0.001–1e6 mm, pixel 0.1–1e6 μm). Display precision was also tightened per field: λ 5→4 decimals, Lsd 4→3, pixel size/BC_y/BC_z 2→1, ty/tz tilts 4→2. "Show rings" was renamed **Rings** and, together with **Labels** and a shrunk ring-thickness field (max width 60px), now sits on one row instead of thickness alone on the row below. **Files:** `midas_gui/tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 2525277`. Self-contained — restores the previous ranges/decimals and the two-row Rings/Labels + Ring-thickness layout.

`b7a1518` — Data Viewer: add "?" help button explaining radial-integration calculation (2026-07-31)

Effect: A small circular **"?"** button now sits next to the radial-profile toolbar's `Radial` control; clicking it opens a message box explaining how the R-bin profile is computed — full-geometry (η, R) binning (pixel-count-weighted mean across η) when a calibration is loaded or a tilt is set, versus the circle-binning fallback (plain per-bin mean, no tilt correction) otherwise, and that a failed full-geometry integration falls back to circle binning with a warning above the calibration card. Also widens the ring-width spinbox (60px → 80px) so its value isn't clipped. **Files:** `midas_gui/tab_view.py`. **Roll back:** `git revert b7a1518`. Self-contained — removes the help button and reverts the ring-width spinbox to 60px.

Rollback recipes (common intents)

- **Undo the Pump Probe tab only:** `git revert 590b410` removes Pump Probe *and* modular tabs. To drop Pump Probe alone, hand-remove `midas_gui/tab_pumpprobe.py`, its import/construction/wiring in `app.py`, `PumpProbeWorker` in `workers.py`, and its entry in `constants.OPTIONAL_TABS` / `DEFAULT_VISIBLE_TABS`.
- **Undo modular tabs (show all 10 fixed again):** remove the `apply_tab_visibility` logic in `app.py` (add all tabs directly), the Tabs section in `prefs_dialog.py`, and the tab-list constants — see the `590b410` diff for the exact hunks.
- **Undo the config system:** `git revert d30be2c` — but first revert `590b410`'s config touches, since it extends the same `ui` overlay and Preferences dialog.
- **Revert the azimuthal-averaging change:** don't blanket-revert `41f28f5` (bundled with UI); instead set the `weighted` default back to the legacy η-bin mean in the batch/pump workers.
- **Go back to the pre-3-panel single-column tabs:** `aa9395e` is the pivot; a revert is large and cascades to later tabs — prefer fixing forward.

Generated from `git log`. To refresh after new commits: `git log --reverse --stat --date=short` and append entries above.